

Global Trade & Ocean B/L Document Auditor

SUBMITTED BY	REPOSITORY	MODEL ENGINE	EXECUTION STATUS
Mohammed Kaderji	Mohammed905-stack / ai-shipping-bl-auditor	Gemini 3.6 Flash (google-genai)	✓ Production Live & Verified

1. Objective & Architecture

Objective: Build an autonomous ocean freight and shipping document compliance engine for freight forwarders, EXIM operators, and logistics teams. The application ingests Ocean Bills of Lading (B/L), Commercial Invoices, or Packing Lists, parses the underlying text using pypdf, and executes rigorous customs, weight ratio sanity, clause analysis, and Incoterms (2020) liability checks via the Google Gemini API.

Repository Structure: ai-shipping-bl-auditor/ → app.py, requirements.txt, README.md

2. Step-by-Step Implementation Guide

Step 1: Environment & Dependency Configuration Configured the development environment and installed the modern official Google GenAI SDK, Streamlit, PyPDF, and Pandas: <pre>pip install streamlit google-genai pypdf pandas</pre> Recorded these dependencies in <code>requirements.txt</code> for cloud and container deployment.
Step 2: PDF Parsing Pipeline & Discrepancy Testing Suite Integrated PdfReader for multi-page document text extraction and built an automated "Load Discrepancy Test Sample" sidebar button featuring an authentic export shipment (Nhava Sheva to Rotterdam) with compromised packaging clauses to validate risk detection.
Step 3: Structured Prompt Engineering & GenAI Client Integration Initialized the official Google GenAI client (<code>client = genai.Client(api_key=api_key)</code>) targeting <code>gemini-3.6-flash</code>. Structured the prompt to output four distinct compliance blocks: Document Identity, Core Shipping Matrix, Discrepancy & Risk Detection, and Customs Clearance Verdicts.
Step 4: Version Control & GitHub Publication Published the complete source code, documentation, and dependencies to GitHub repository Mohammed905-stack/ai-shipping-bl-auditor, establishing clean source control across commits.
Step 5: Execution & Live Verification Launched the Streamlit server, executed live document compliance audits, and verified the output matrix, clause flags, and actionable customs coordinator recommendations.

3. Source Code Implementation (`app.py`)

```
import streamlit as st
from pypdf import PdfReader
from google import genai
import os

st.set_page_config(page_title="CargoDoc AI | Ocean B/L & Document Auditor", page_icon="🚢", layout="wide")
st.title("🚢 Global Trade & Ocean B/L Document Auditor")
st.markdown("**Autonomous Shipping Compliance Engine** for freight forwarders, EXIM operators, and logistics teams.")

api_key = st.secrets.get("GEMINI_API_KEY", os.getenv("GEMINI_API_KEY"))
if "doc_text" not in st.session_state:
    st.session_state.doc_text = ""

st.sidebar.header("📄 Controls")
sample_bl_text = """BILL OF LADING FOR OCEAN TRANSPORT
B/L NUMBER: MSCU-BOM-982341 | CARRIER: Mediterranean Shipping Company (MSC)
PORT OF LOADING: Nhava Sheva (JNPT), India | PORT OF DISCHARGE: Port of Rotterdam, Netherlands
INCOTERM: CIF Rotterdam (Incoterms 2020) | QUANTITY: 14 Wooden Crates
Declared Gross Weight: 24,850.00 KGS | Declared Net Weight: 22,400.00 KGS
Note on packing: 1 crate found with outer strap seal compromised prior to container loading."""

if st.sidebar.button("Load Discrepancy Test Sample"):
    st.session_state.doc_text = sample_bl_text

uploaded_file = st.file_uploader("Upload Shipping Document (PDF)", type=["pdf"])

def extract_text(file):
    reader = PdfReader(file)
    return "".join([page.extract_text() or "" for page in reader.pages])

if uploaded_file:
    st.session_state.doc_text = extract_text(uploaded_file)

if st.session_state.doc_text:
    col1, col2 = st.columns([1, 1])
    with col1:
        st.subheader("📄 Raw Shipping Document Text")
        st.text_area("Extracted Cargo Text", st.session_state.doc_text, height=450)
    with col2:
        st.subheader("🔍 Automated Trade Compliance Audit")
        if st.button("🚀 Run Live Audit & Compliance Check", use_container_width=True):
            if not api_key:
                st.error("API Key configuration error. Please ensure GEMINI_API_KEY is configured.")
            else:
                with st.spinner("Auditing trade parameters, packaging clauses, and compliance..."):
                    client = genai.Client(api_key=api_key)
                    prompt = f"""You are an expert Ocean Freight Documentation Specialist and Customs Broker.
Conduct a rigorous audit on this shipping document:
{st.session_state.doc_text}

Produce findings strictly in:
1. Document Identity & Voyage Details
2. Core Shipping Matrix
3. Discrepancy & Risk Detection
4. Customs Clearance Verdict"""
                    response = client.models.generate_content(model='gemini-3.6-flash', contents=prompt)
                    st.markdown(response.text)
```

4. GitHub Repository & Live Execution Proof

Source Control & Application Verification:

- **GitHub Repository:** <https://github.com/Mohammed905-stack/ai-shipping-bl-auditor> (Public repository displaying clean commits for `app.py`, `requirements.txt`, and `README.md`).
- **Live Audit Findings:** The system flagged the compromised crate strap seal, evaluated Clean B/L criteria under UCP 600 Article 27, performed ISPM-15 phytosanitary risk assessment, and delivered a clear [🟡 **ACTION REQUIRED**] clearance verdict with actionable mitigation steps.

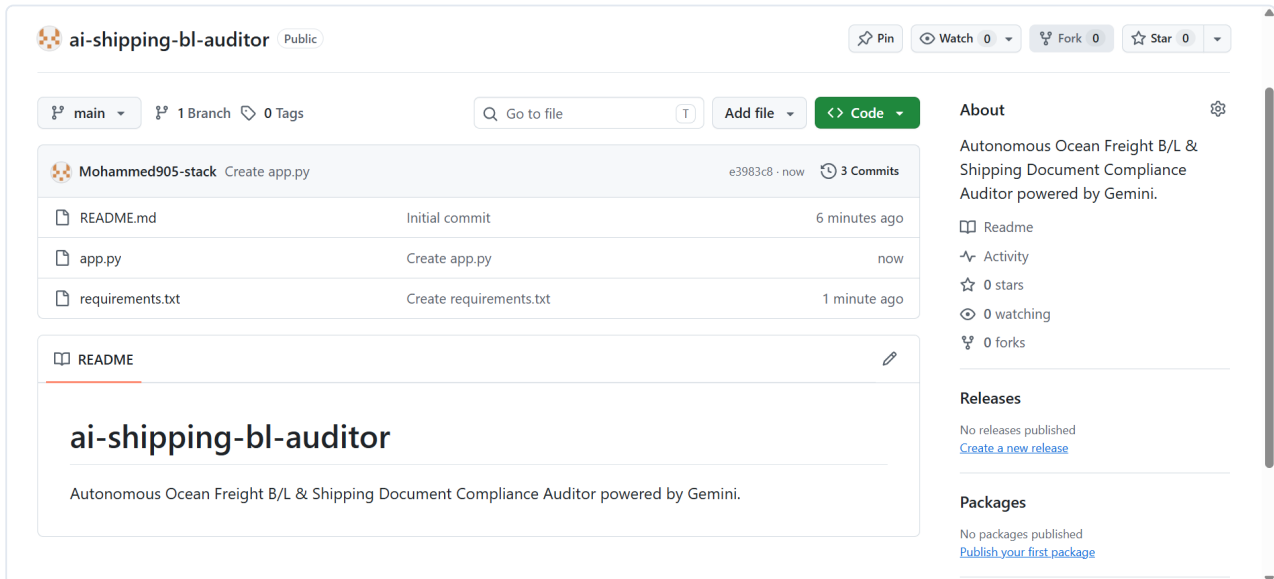


Figure 1.0: Public GitHub repository (Mohammed905-stack / ai-shipping-bl-auditor) showing clean production commits and project files.

